

BJPG-01背景评估

- 1.产品列表
- 2.开户步骤
- 3.工具类（用来加密入参，解密出参）
- 4.请求举例
 - 4.1 请求头
 - 4.2 请求体
 - 4.3 返回体

1.产品列表

序号	产品编号	产品名称	产品描述	备注
1	BJPG-01	背景评估		

2.开户步骤

找 对应接口人 申请账户，开通后将获取如下信息：

```
1  测试环境：
2      name: xx科技-test
3      accountId: ds20230205xxxx
4      encryptKey: 0031cee6808eb6d5b0e07536218fxxxx (该值用来加密解密)
5      产品列表: BJPG-01 (举例)
6      接口地址: http://122.224.147.153:13199/api/getData
7
8
9  生产环境：
10     name: xx科技-prod
11     accountId: ds20230205xxxx
12     encryptKey: 0031cee6808eb6d5b0e07536218fxxxx (该值用来加密解密)
13     产品列表: BJPG-01 (举例)
14     接口地址: http://122.224.147.153:13188/api/getData
15
```

3.工具类（用来加密入参，解密出参）

```
1  import java.nio.charset.StandardCharsets;
2  import javax.crypto.Cipher;
3  import javax.crypto.spec.IvParameterSpec;
4  import javax.crypto.spec.SecretKeySpec;
5  import org.apache.commons.codec.binary.Base64;
6  import org.apache.commons.codec.binary.Hex;
7
8  /**
9   * @author xxx
10  */
11  public final class AesUtil {
12
13      public AesUtil() {
14      }
15
16      /**
17       * 加密
18       *
19       * @param content String
20       * @param key      String
21       * @return String
22       */
23      public static String encode(String content, String key) {
24          try {
25              SecretKeySpec keySpec = new SecretKeySpec(Hex.decodeHex(key.toCharArray()), "AES");
26              System.out.println("keySpec:" + keySpec);
27              Cipher c = Cipher.getInstance("AES/CBC/PKCS5Padding");
28              c.init(1, keySpec, new IvParameterSpec("0000000000000000".getBytes()));
29              return Base64.encodeBase64String(c.doFinal(content.getBytes(StandardCharsets.UTF_8)));
30          } catch (Exception var4) {
31              var4.printStackTrace();
32              return "";
33          }
34      }
35
36      /**
37       * 解密
38       *
39       * @param content String
40       * @param key      String
41       * @return String
42       */
43  }
```

```

43     public static String decode(String content, String key) {
44         try {
45             SecretKeySpec keySpec = new SecretKeySpec(Hex.decodeHex(key.toCharArray()), "AES");
46             Cipher cipher = Cipher.getInstance("AES/CBC/PKCS5Padding");
47             cipher.init(2, keySpec, new IvParameterSpec("0000000000000000"
48                 .getBytes()));
49             byte[] encryptAesResByte = Base64.decodeBase64(content);
50             byte[] responseByte = cipher.doFinal(encryptAesResByte);
51             return new String(responseByte);
52         } catch (Exception var6) {
53             var6.printStackTrace();
54             return "";
55         }
56     }

```

4.请求举例

4.1 请求头

请求头中加上accountId(账户id)和prodId (产品id) ， 例如请求BJPG-01产品数据

Params	Authorization	Headers (11)	Body	Pre-request Script	Tests	Settings
☒	User-Agent				① PostmanRuntime/7.32.2	
☒	Accept				① */*	
☒	Accept-Encoding				① gzip, deflate, br	
☒	Connection				① keep-alive	
☒	accountId				10000001010011131	
☒	prodId				1	
	Key				Value	Description
Response						

4.2 请求体

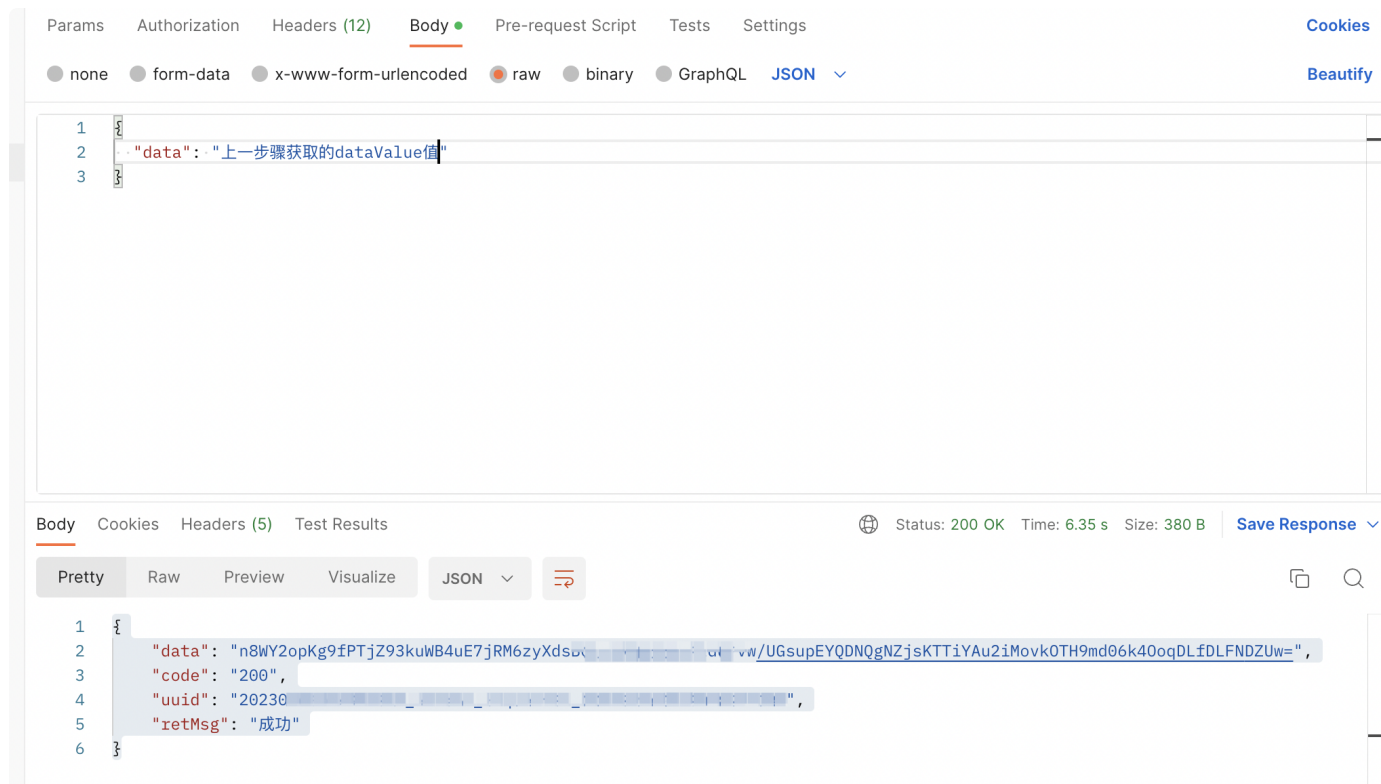
1.先查看对应产品服务文档，那些字段是必填。

```
Java |  
1 idCard: String 身份证号码，必填  
2  
3 name: String 姓名，必填
```

2.使用加密工具类，对入参进行加密

```
Java |  
1 String encryptKey = "自己账户的encryptKey值";  
2 Map<String, Object> params = new HashMap<>();  
3 params.put("idCard", "xxx");  
4 params.put("name", "xxx");  
5 String dataValue = AesUtil.encode(JSON.toJSONString(params), encryptKey);  
6 System.out.println(dataValue);
```

3.将上面步骤的dataValue值放入请求body中



4.3 返回体

1.返回体的大致格式如下：

▼ Java

```
1  {    // 只有code=200, data才有值
2      "data": "responseData, 返回的是一个加密字符串, 需要用AesUtil.decode解密",
3      "code": "200", //200是成功
4      "uuid": "xxx",
5      "retMsg": "成功"
6
7
8
```

代码	描述	是否计费
200	请求成功	计费
2-404	没有查询到数据	不计费
2-500	服务器内部异常	不计费
2-501	解密失败, 请检查data加密方式	不计费
2-502	参数不全或者参数不正确	不计费
2-503	请求token失败	不计费
2-504	您没有该接口的访问权限, 请联系管理员处理	不计费
2-505	今日调用次数超过限额	不计费
2-506	请求头中缺少accountId	不计费
2-507	请求头中accountId不正确	不计费
2-508	请求ip不在白名单内, 请联系管理员	不计费
2-509	账户已经停止使用, 请联系管理员	不计费
3-510	查询失败	不计费
3-404	没有查询到	不计费
3-506	权限问题	不计费

3-507	获取数据失败	不计费
-------	--------	-----

2.解密结果

Java

```
1 String jsonStr = AesUtil.decode("上面步骤的responseData","自己账号的encryptKey")
2
3 例如解密后结果为字符串:
4 {"jfsj":"202603","cbjfzt":"1","jfjs":"6800","xm":"石磊","sfz":"430481199",
5  "jfdw":"珠海科技有限公司","grsf":"24"}
6
7
8
9
```

序号	字段名	字段类型	字段含义	备注
1	xm	String	姓名	
2	sfz	String	身份证	
3	jfdw	String	缴费单位	某公司
4	grsf	String	个人身份	可忽略
5	jfjs	String	缴费基数	
6	cbjfzt	String	参保状态	1：正常参保；2：暂停参保；3：停止参保；
7	jfsj	String	缴费时间	yyyymm格式，例：202401